

One Global Beam Across Many GPUs: Exact High-Throughput Beam Search at Billion-State Frontier Scale

Ivan Litvak
Independent researcher
vanya.litvak@gmail.com

Abstract

We present the first published high-throughput multi-GPU beam-search system that physically shards one logical frontier across devices and jointly constructs one next beam. Candidate generation, scoring, 128-bit-key de-duplication, routing, and frontier storage remain on the GPUs; only compact control and ancestry metadata reach the host. With no shard-local semantic caps, the streamed procedure returns the same global reduced-key top- B as a monolithic implementation under the same score quantization, fixed-layout tie order, and stated 128-bit identity model. On eight H200 GPUs, a Cube4 run completed one saturated depth with a physically sharded retained frontier of $B_{\text{eff}} = 2,900,361,216$ states in 931.266 s. With 24 generators, this is 69,608,669,184 nominal pre-de-duplication parent-action pairs, or 74.746 million pairs/s end to end. The run was stopped during the next depth and did not solve the puzzle. Separately, a measured two-T4 run at $B_{\text{eff}} = 82,837,504$ has 1,988,100,096 nominal children per saturated depth; depths 7–10 averaged 65.670 s, or 30.274 million logical children/s end to end. These runs use different tasks and models, so they do not establish strong scaling. The novelty is the full system-level conjunction, not parallel, GPU, or distributed beam search in isolation.

1 Introduction and Problem

Beam search becomes a distributed-systems problem once the retained frontier exceeds one GPU’s memory. Expansion, scoring, de-duplication, global selection, and ancestry must be distributed without turning one search into independent replicas or staging the frontier through host memory. We ask whether a GPU cluster can preserve the semantics of one monolithic beam while scaling retained capacity with aggregate device memory.

Let $G = (V, E)$ be an implicit directed graph, $A_a(x)$ the child of state x under generator $a \in \{1, \dots, m\}$, and $\sigma(x, a)$ a deterministic floating-point scoring oracle. The mathematical contract and Streams 2–5 permit a learned or hand-engineered oracle provided Stream 1 emits deterministic quantized keys; the shipped production Stream 1 backends and reported experiments are neural. The implementation ranks only the clipped and quantized integer

key $s(x, a)$ defined in Eq. (4). A scalar state scorer is the special case $\sigma(x, a) = q(A_a(x))$, where $q : V \rightarrow \mathbb{R}$; an output- m scorer emits all action scores from one parent evaluation. At depth d , beam search computes

$$\begin{aligned} \mathcal{C}_d &= \{(x, a, A_a(x)) : x \in \mathcal{F}_{d-1}, a \in [m]\}, \\ U_d &= \text{Reduce}_z(\mathcal{C}_d; \text{lexmin}(s, \text{payload})), \end{aligned} \quad (1)$$

$$\mathcal{F}_d = \text{Top}_B(U_d; s_d^*, \tau_L), \quad (2)$$

where z is the 128-bit child-state key, $s_d^*(z) = \min\{s(x, a) : z(A_a(x)) = z\}$, and τ_L is the deterministic tie order induced by a fixed rank/shard layout L . Thus de-duplication keeps the best candidate for each key before the global cut. For a saturated layer in which every retained parent is expanded by all m generators, the nominal pre-de-duplication count is mB_{eff} . Since runtime alignment gives $B_{\text{eff}} \geq B_{\text{req}}$, the measured H200 frontier $B_{\text{eff}} = 2,900,361,216$ with $m = 24$ implies 69,608,669,184 nominal logical children in one saturated depth. Materializing its full 96-byte Cube4 child states would require over 6.68 TB before metadata; the design avoids that host staging and any single-device global sort.

Our requirement is stronger than GPU-accelerated heuristic evaluation. During one depth, the candidate and frontier arrays in Eq. (2) remain in device memory. The host receives only $O(p)$ control counts and compact selected-candidate metadata, never the full state or frontier arrays. Here “one coherent search” has an operational meaning: every GPU contributes to Eq. (2), and the ranks collectively commit disjoint shards of one globally defined next frontier. It is not an ensemble of independent beams or replicas.

Contributions. The system contributes (i) a GPU-resident hot-data path that distributes one logical beam without frontier staging through host memory; (ii) exact monolithic global reduced-key top- B semantics under the same score quantization, fixed-layout tie order, and stated 128-bit identity model, with no shard-local semantic caps; and (iii) a measured saturated billion-state frontier depth on eight H200 GPUs, with a separate two-T4 production result. Together, these contributions make this the first published high-throughput multi-GPU beam-search system with one physically sharded global frontier and the stated exactness contract. The implementation admits at

most 128 ranks; execution beyond eight GPUs and strong scaling at a fixed workload remain unmeasured.

2 Related Work and Gap

Parallel beam search long predates GPUs: custom speech accelerators were studied in 1986 [2]. Wijs and Lissers later distributed a single global beam over CPUs [19], and Chong et al. placed a data-parallel speech beam on one GPU [6]. Thus, neither parallel, distributed, nor GPU beam search is new by itself. Modern CPU work retains beams up to 20 million on a shared-memory machine, while MPI workers run independent randomized beams [7]; Kuroiwa and Beck also study global and locally partitioned CPU variants [12].

For GPU and heuristic combinatorial search, GA* establishes single-GPU A* without a neural scorer [21], GPUAS-tar keeps neural A* on one H200 [14], and HDA* establishes hash-distributed CPU best-first search [11]. CayleyPy is the closest algorithmic predecessor: its large-cube agents use one GPU and $B = 2^{24}$ [5, 13]. AlphaCube and DeepCubeA are single-device beam search and batched weighted A*, respectively [18, 1].

The closest adjacent systems result is QiankunNet-cuSCI, which generates configurations, performs distributed global de-duplication, applies a neural Transformer, and selects top- K on 64 A100 GPUs with 91.06% reported strong-scaling efficiency [17]. It prevents a claim to the generic expand–deduplicate–infer–select pipeline, but the work is not reported as beam search. It retains $K = 32,000$ or 256,000 selected SCI configurations from roughly one billion raw candidates, and its global barriers stage full candidate data through host memory.

Other billion-scale results measure different objects. GPUexplore 3.0 explores more than 3.15 billion reachable states on four GPUs but performs exhaustive exploration without score-ranked bounded top- B [20]. PathWeaver pipelines approximate nearest-neighbor queries over a stored graph [10]; multi-GPU speech and language decoders distribute independent utterances, sentences, or model execution around tiny beams rather than physically sharding one state frontier [9, 3, 4].

Table 1 applies one scale taxonomy: retained beam width is not raw candidates, cumulative expansions, graph size, or a batch of independent queries. Prior systems establish important subsets of parallel beam search, GPU execution, distributed search, or large-scale state exploration, but do not report their full conjunction in one system. This work is the first published high-throughput multi-GPU beam-search system that combines one physically sharded logical frontier, a GPU-resident candidate/frontier hot-data path, global 128-bit-key de-duplication, no shard-local semantic caps, and exact monolithic global reduced-key top- B under the stated quantization, tie order, and identity model at a measured billion-state retained frontier. The priority claim applies only to this full system-level conjunction.

3 One Search Depth on Five GPU Streams

The input is a global beam $\mathcal{F}_{d-1}$ distributed over p GPUs. If GPU r owns N_r parents and the graph has m moves, it generates mN_r logical candidates. For $y = A_a(x)$, a candidate is represented before materialization by

$$c = (z(y), s(x, a), i_{\text{parent}}, a, r_{\text{src}}), \quad (3)$$

where $z(y) \in \{0, 1\}^{128}$ is the child-state hash, $s(x, a)$ is a quantized candidate score, and r_{src} identifies the GPU that stores the parent. The child state itself is reconstructed only for goal checking or after the final selection.

Streams 1–2: score and identity. Parents are divided into microbatches and placed in reusable ring slots. Streams 1 and 2 consume the same slot concurrently. Stream 1 evaluates the chosen scoring oracle and writes only

$$s(x, a) = \text{rint}_{\text{RN-even}}(S \text{ clip}(\sigma(x, a), 0, q_{\text{max}})) \in \mathbb{N}, \quad (4)$$

not the floating-point output. Here RN-even is the GPU path’s default round-to-nearest, ties-to-even mode. The scalar path evaluates child states, whereas the output- m path obtains all m action scores from each parent. Stream 2 applies every move in registers, checks the goal, and writes the 128-bit Zobrist hash. Thus score evaluation and state manipulation overlap without storing the mN_r child states.

Stream 3: early reduction and ownership. When a group of ring slots has both scores and hashes, Stream 3 removes scores above the current threshold T , compacts the survivors, sorts them by z , and reduces equal hashes. For each hash it retains the lexicographically smallest pair $(s, \text{payload})$, where the payload recovers the original parent and move. It then assigns the unique candidate to

$$o(x) = \text{mix}(z(x)) \bmod p. \quad (5)$$

Local candidates enter the local collector; the rest are grouped by owner. Performing this first reduction before communication lowers traffic, but does not replace the later global deduplication.

Stream 5: GPU-to-GPU exchange. Stream 5 exchanges the owner-grouped metadata directly between GPUs using NCCL. The owner rule implies

$$z(x) = z(y) \implies o(x) = o(y), \quad (6)$$

so duplicates produced by different source GPUs eventually meet at one owner. Stream 5 performs communication only; received candidates re-enter the same collector as local candidates.

Table 1: Closest prior systems under a common scale taxonomy. “Other scale” is deliberately not interpreted as retained beam width.

System	Platform and search object	Retained live beam	Other scale	Missing from the claimed conjunction
Wijs-Lisser 2007	distributed CPU; one global beam	50,000	135.96M cumulative states	GPU data plane and device-resident scorer
Chong et al. 2008	one 8800 GTX; speech active set	20,000 cap	1M-state model	multi-GPU state frontier
Frohner et al. 2022	46 CPU threads; one shared-memory beam	20,000,000	MPI: independent beams	GPU and distributed one-beam execution
CayleyPy 2025	one GPU; one Cayley-graph beam	2^{24}	independent agents	multi-GPU global frontier
GPUexplore 3.0	four V100; exhaustive reachable set	not a beam	$> 3.15\text{B}$ states	ranked score and top- B
PathWeaver 2025	four A6000; independent ANNS queries	not reported	50M-vector dataset	exact layered semantics and large retained beam
QiankunNet-cuSCI	64 A100; selected SCI configurations	not a beam; $32,000/256,000$	$K =$ about 1B raw candidates	not reported as beam search; much smaller retained K ; full-candidate host staging
This work	eight H200; one sharded global beam	2,900,361,216 effective	69.609B nominal raw pairs/depth	completed depth 8; full solve and fixed-workload strong scaling unmeasured

Stream 4: persistent survivor set. The collector partitions owned candidates into logical shards. Each shard has two resident buffers: Stream 3 appends to one while Stream 4 cleans the other. A Stream 4 job merges the clean and newly appended regions, applies its snapshot of T , compacts, sorts by z , and retains the best metadata for each hash. There is no per-shard top- k and no semantic shard quota: a shard may retain every candidate below T . Consequently sharding does not approximate the score cut; only the equal-score boundary order is defined by the fixed layout.

Classical top- B versus monotone thresholding. Let C be the complete candidate multiset, write $s(c)$ for the quantized integer score in Eq. (4), and let $U = \text{Reduce}_z(C; \text{lexmin}(s, \text{payload}))$ be its one-entry-per-key reduction, carrying $s^*(z) = \min_{c: z(c)=z} s(c)$ and the associated best payload. For fixed layout L , rank keys by $\kappa_L(z) = (s^*(z), \tau_L(z))$, where smaller score is better and τ_L is the implementation’s deterministic rank-then-physical-order rule for equal scores. A classical reference materializes U , sorts it by κ_L , and returns, for $b = \min(B, |U|)$,

$$\text{Top}_B(U) = \{z \in U : \kappa_L(z) \leq \kappa_{L,(b)}(U)\}. \quad (7)$$

Our implementation never materializes U . After update j , let $C_j \subseteq C_{j+1} \subseteq C$ be the candidates processed so far and $U_j = \text{Reduce}_z(C_j)$. The key domain of U_j can only grow, while an existing key score $s_j^*(z)$ can only decrease. Once $|\text{dom } U_j| \geq B$, histograms of the current reduced scores define

$$\begin{aligned} \hat{T}_j &= \min \left\{ t : \sum_{u=0}^t H_j(u) \geq B \right\}, \\ T_j &= \min\{T_{j-1}, \hat{T}_j\}, \quad T_0 = +\infty. \end{aligned} \quad (8)$$

While $|\text{dom } U_j| < B$, $\hat{T}_j = +\infty$. Streams 3 and 4 may discard only candidates with $s(c) > T_j$; the score boundary

$s(c) = T_j$ is retained because its final membership depends on τ_L .

Exactness. Processing another candidate either introduces a key or lowers an existing key’s best score, so the B -th smallest reduced score cannot increase; hence $\hat{T}_{j+1} \leq \hat{T}_j$ and $T_{j+1} \leq T_j$. If candidate c is discarded at update j , then $s(c) > T_j$ and U_j already contains at least B distinct keys with current best scores at most T_j . Those keys remain distinct and their final best scores can only decrease. If $z(c)$ is among them, c is worse than its stored representative; otherwise at least B other keys are strictly better. Thus c cannot determine a selected final minimum. Conversely, a candidate attaining the final minimum of a global top- B key cannot be discarded, because that would require B distinct keys already carrying strictly smaller scores. Thresholding therefore preserves exactly the classical reduced-key score top- B under the same quantization for fixed L , subject to the same fixed-layout tie rule at the boundary.

Identity and exactness scope. The proof is exact for the score quantization in Eq. (4), the set keyed by the 128-bit Zobrist identifier, and the complete fixed-layout order κ_L . The candidate data plane does not carry all 120 logical state bytes, so equal 128-bit keys are treated as equal states rather than resolved by a full-state comparison. Selection is therefore mathematically exact under this stated key model; the residual risk is a 128-bit hash collision, not an approximation introduced by sharding, thresholding, or top- B . Changing the GPU or shard layout can change τ_L and therefore which equal-score states occupy the boundary; the current implementation does not claim partition-invariant selected IDs.

Final cut and materialization. After all five streams are drained, the ranks combine the final histograms. If $|U| \geq B$, they choose the smallest T^* such that

$$\sum_{u=0}^{T^*} H(u) \geq B. \quad (9)$$

All states below T^* survive, and a deterministic tie-break cuts its boundary to exactly B states. If $|U| < B$, all available keys survive instead. Thus the next frontier always contains exactly $\min(B, |U|)$ states. The selected metadata are load-balanced across GPUs. Only then does each source GPU reconstruct the requested children and send them directly to their target GPUs. The CPU receives $O(p)$ count vectors plus compact selected-candidate metadata used to reconstruct ancestry, but no full state or frontier arrays.

The five streams are readiness-driven rather than five serial phases. Streams 1–2 fill ring slots almost continuously while Streams 3 and 4 reduce earlier batches and Stream 5 exchanges remote survivors. The mathematical invariant is that every hash surviving the final cut is reduced at its unique owner before Eq. (9); hence, for a fixed layout, the distributed execution implements the same global reduced-key score top- B as its monolithic reference under the same score quantization, without materializing the raw candidate layer.

4 Experiments

Protocol. We evaluate computational cost rather than solution quality and report a measurement only with its hardware, workload, model contract, and scope. Production time includes generation, de-duplication, communication, selection, and history; it is not interchangeable with an isolated Stream 1 rate. Empty cells mean “not measured”, never an inferred value. Table 2 summarizes the strongest locally verified measurements.

In Table 2, the RTX value is total E2E wall divided by 20 completed depths, whereas the Kaggle value is the summary-reported mean of saturated depths 7–10. The T4 CSV records both requested and aligned effective widths; 30.274 M/s is the explicit derivation $24 \times 82,837,504 / 65.670$ and counts logical children before de-duplication. We do not derive an RTX E2E rate because its early depths are not saturated. Kernel microbenchmarks and the A100 capacity-only summary do not define an E2E depth time and are therefore marked not applicable or not recorded. The H200 rate is the nominal $24B_{\text{eff}}/931.266$; it counts parent-action pairs, not distinct children after de-duplication.

Local RTX 3070. On the laptop GPU, the folded-input half2 kernel raised the best isolated Stream 1 rate from approximately 24.7 to 48.142 million candidates/s. In Nsight, the changed kernel fell from 9.678 to 3.232 s over 1,202 launches (2.99 $\times$). More importantly, the same production

depth-20 workload fell from 115.334 to 60.623 s, a 1.90 $\times$ end-to-end improvement. A legacy PyTorch run at depth 100 exhausted memory at depth 33 after 329.067 s; we do not count this incomplete run as a speedup baseline.

Kaggle 2 $\times$ T4. The two ranks jointly maintain one global beam. For puzzle 20, requested $B_{\text{req}} = 82,615,524$ was aligned to $B_{\text{eff}} = 82,837,504$ and the radius-4-neighborhood run solved in 350.402 s with 128 shards and $B_{\mu} = 4096$. At $B = 2^{26}$, increasing the shard count from 8 to 64 reduced the mean time of saturated depths 7–9 from 69.365 to 51.732 s (1.34 $\times$) while retaining exact global selection. The best isolated Stream 1 point was 40.60 M candidates/s; it is not used as an E2E rate.

8 $\times$ A100 capacity and scale-out limit. The exact-scale autotune summary records stable depth-8 execution on three Megaminx instances for requested $B_{\text{req}} = 770,883,178$, peaking at 39,745 MiB per GPU. Runtime alignment makes $B_{\text{eff}} \geq B_{\text{req}}$, but the exact effective value, raw rank logs, runtime configuration, commit, checkpoint hash, and wall time were not preserved. The summary therefore supports an eight-device capacity result and a nominal saturated scale of at least 18.501 billion logical child candidates, not a timed A100 result. The separate H200 run below provides eight-GPU performance on a different puzzle. Static exchange tables currently admit at most 128 ranks, but execution beyond eight GPUs is unmeasured.

8 $\times$ H200 billion-state depth. On one Vast instance with eight H200 GPUs (143,771 MiB each), the Cube4 24-generator puzzle-1000 run requested $B = 2,900,000,000$ and aligned to $B_{\text{eff}} = 2,900,361,216$, or 362,545,152 retained states per rank. All eight ranks completed saturated depth 8 in 931.266 s. This is 3.114 million parents/s or 74.746 million nominal parent-action pairs/s end to end, including scoring, global reduction, exchange, selection, and history handling. The scorer was the 24-output, 3,383,064-parameter FP16 Cube4 piece Transformer (four 256-wide, eight-head blocks; 1024-wide feed-forward), with model-manifest SHA-256 prefix 03f55e5b (full digest in the evidence manifest). The implementation revision was 3b756afe; the run used 64 shards per GPU and device-wide usage of about 141,820 MiB/GPU. At 2.94 billion requested states the first NCCL count exchange failed; 3.00 billion failed a static memory-budget gate. Neither is classified as CUDA OOM. Puzzle 1000 was intentionally stopped during depth 9, so depth 8 is a completed-step timing, not a complete solve. The Cube4 workload and model differ from the Megaminx two-T4 run; the two rates are not a strong-scaling comparison.

Table 2: Measured performance by platform. E2E is the production critical path; micro is an isolated kernel pipeline; capacity has no timing claim. n/d: not derived; n/a: not applicable; n/r: not recorded.

Platform	Workload	Beam B	Measurement	s/depth	Raw rate	child
RTX 3070 8GB	puzzle 0, depth 20	2^{22}	E2E complete; 60.623 s total	3.031 avg.	n/d	
RTX 3070 8GB	Stream 1, $B_\mu = 4096$, $c = 3$	—	micro complete	n/a	48.142 M/s	
Kaggle 2×T4	puzzle 20, radius 4, limit 72	$B_{\text{req}} = 82,615,524$; $B_{\text{eff}} = 82,837,504$	solved; 350.402 s; CSV+summary	65.670 rated	30.274 M/s	
8×A100 40GB	puzzles 900/950/1000, depth 8	$B_{\text{req}} = 770,883,178$; B_{eff} not recorded	stable; 39,745 MiB/GPU; summary	n/r	n/r	
8×H200	Cube4 puzzle 1000, depth 8	$B_{\text{req}} = 2,900B$; $B_{\text{eff}} = 2,900,361,216$	E2E depth complete; $\sim 141,820$ MiB/GPU used	931.266	74.746 M/s	

Table 3: Published CayleyPy beam-width trend (Table 8 of [13]).

W	2^{10}	2^{12}	2^{14}	2^{16}	2^{20}
Length	—	252	202	194	190

Table 4: Matched Cube4 depth-10 computation at $B = 4,194,304$.

GPU	Implementation	s/depth (depth 10)	Pairs/s	Peak VRAM
T4	Pilgrim	198.153	508,007	2.90 GiB reserved
T4	Ours	181.282	555,293	1,949 MiB used
P100	Pilgrim	314.487	320,087	2.90 GiB reserved
P100	Ours	301.955	333,378	1,949 MiB used

5 Comparison with CayleyPy and Megaminx Result

CayleyPy establishes empirically that wider beams improve success and solution quality, but its published examples remain single-device. Its controlled S_{20} experiment increases W from 2^{10} (no solution) to 2^{20} (the ideal length 190), while the large-cube paper uses $W = 2^{24}$ so that an agent fits on one GPU [13, 5]. The measured H200 retained width of 2,900,361,216 is $172.875\times$ wider than 2^{24} , $145.02\times$ wider than the largest peer-reviewed retained beam located in our audit (20 million), and $116.01\times$ wider than the 25-million public HATETRIS implementation [8]. These ratios compare retained capacity, not speed or solution difficulty across different graphs.

We additionally measured Pilgrim and our search implementation on the same Cube4 state, checkpoint, single GPU, beam, and ten layers. The scorer is a 3,383,064-parameter piece Transformer with 96 six-class positions, $d_{\text{model}} = 256$, four layers, eight heads, feed-forward width 1024, and 24 Q outputs; inference uses FP16 and micro-batches of 384. Table 4 reports the tenth layer, where every saturated frontier expands $24B$ pairs.

5.1 Native approach inventory and matched model families on one T4

Table 6 is an inventory, not a cross-row speed ranking. AlphaCube and DeepCubeA retain their released Cube3 models. Megaminx results form two disjoint matched families: scalar `output_dim=1` (Pilgrim Searcher, native CayleyPy, MultiGPUBeamSearch) and native `output_dim=24` (Pilgrim QSearcher, MultiGPUBeamSearch). Ratios require the identical checkpoint, architecture, head, state, generators, depth, width, dtype, and hardware. CayleyPy cannot consume the 24-output head and remains incompatible rather than transformed. The rate column contains only native counted nodes divided by solver wall time; it is never replaced by B/t .

For the matched Megaminx rows, all listed implementations load the same bytes: the scalar checkpoint has SHA-256 `7f5071e6...a539b759`, while the output-24 checkpoint has SHA-256 `e7bda332...5d2ca670`. The latter is not derived from the scalar model. The Cube4 Transformer is a separate matched comparison and is never mixed into the Megaminx capacity sweep.

For AlphaCube, s/depth is solver wall divided by the completed search depth reported by the returned solution (20 for the first three rows and 17 for the remaining rows). For both Megaminx implementations it is wall divided by the fixed eight completed depths. These averages include startup and early unsaturated layers; they complement, rather than replace, nodes/s.

Pilgrim’s widest scalar completion, $B = 5,898,240$, took 1,462.557 s (182.820 s/depth) with a 14,433 MiB device peak; $B = 6,291,456$ CUDA-OOMed. For output-24, $B = 20,971,520$ completed in 197.358 s (24.670 s/depth) at 12,333 MiB, whereas $B = 23,068,672$ CUDA-OOMed. The measured brackets are therefore 5,898,240–6,291,456 and 20,971,520–23,068,672, respectively.

The final collaborator-run MultiGPU pass and scalar-only v27 confirmation now give measured complete/OOM pairs for both heads. Scalar completed at 29,360,128 and CUDA-OOMed at 31,457,280; native output-24 completed through 29,360,128 and CUDA-OOMed at 33,554,432.

The scalar OOM recurred for every row-budget profile

Table 5: Exact neural architectures used by the reported tests. Parameter counts refer to the trainable source model before batch-normalization folding.

Model family	Parameters	Used by	Ordered block sequence
Cube3 AlphaCube 0.1.6 large	118,939,666	AlphaCube	one-hot 324 $\rightarrow$ 8 $\times$ [Linear 4096, ReLU, BN] $\rightarrow$ Linear 18
Cube3 DeepCubeA official cube3	14,663,001	DeepCubeA	one-hot 324 $\rightarrow$ Linear 5000, BN, ReLU $\rightarrow$ Linear 1000, BN, ReLU $\rightarrow$ 4 $\times$ [Linear 1000, BN, ReLU, Linear 1000, BN, residual add, ReLU] $\rightarrow$ Linear 1
Megaminx scalar MLP	34,766,849	Pilgrim Searcher; CayleyPy; MultiGPU	position-class 120 $\times$ 120 $\rightarrow$ EmbeddingBag-equivalent Linear 2048, BN, ReLU $\rightarrow$ Linear 512, BN, ReLU $\rightarrow$ 8 $\times$ [Linear 512, BN, ReLU, Linear 512, BN, residual add, ReLU] $\rightarrow$ Linear 1
Megaminx QMLP2RB	23,978,008	Pilgrim QSearcher; MultiGPU	position-class 120 $\times$ 120 $\rightarrow$ EmbeddingBag-equivalent Linear 1536, BN, ReLU $\rightarrow$ Linear 512, BN, ReLU $\rightarrow$ 2 $\times$ [Linear 512, BN, ReLU, Linear 512, BN, residual add, ReLU] $\rightarrow$ Linear 24
Cube4 Q Transformer	3,383,064	matched Cube4 only	96 tokens / 6 classes $\rightarrow$ projection + CLS $\rightarrow$ 4 $\times$ [pre-LN, 8-head attention ($d = 256$), residual, pre-LN, FFN 256-1024-256 ReLU, residual] $\rightarrow$ LN $\rightarrow$ Linear 24

Table 6: Native search inventory on one visible T4. Rows with different model contracts are not performance comparisons; “Max complete” is measured.

Approach	Workload / native scale	Max complete	Wall (s)	s/native step	Nodes/s	Boundary/status
AlphaCube 0.1.6	Cube3 large , beam	1,572,864	461.488	27.146/depth	42,387	1,703,936 host SIGKILL
DeepCubeA	Cube3 A*, 55 iterations	n/a	146.474	2.663/iteration	41,880	complete, length 21
Pilgrim Searcher	Megaminx scalar beam, depth 8	5,898,240	1,462.557	182.820/depth	not measured	6,291,456 OOM
Pilgrim QSearcher	Megaminx output-24 beam, depth 8	20,971,520	197.358	24.670/depth	not measured	23,068,672 OOM
native CayleyPy	Megaminx scalar beam, depth 8	20,971,520	4,258.602	532.325/depth	not measured	broad: 2 ²⁴ complete, 2 ²⁵ OOM
MultiGPUBeamSearch	Megaminx scalar beam, depth 8	29,360,128	2,817.158	352.145/depth	not measured	31,457,280 OOM
MultiGPUBeamSearch	Megaminx output-24 beam, depth 8	29,360,128	116.904	14.613/depth	not measured	33,554,432 OOM

Table 7: AlphaCube 1xT4 completed sweep.

B	Wall (s)	s/depth	Nodes	Nodes/s
1,024	1.034	0.052	18,720	18,108
4,096	1.199	0.060	73,484	61,309
16,384	5.014	0.251	282,380	56,321
65,536	18.543	1.091	903,448	48,722
131,072	37.818	2.224	1,755,416	46,418
262,144	79.166	4.657	3,459,352	43,698
524,288	170.671	10.039	6,867,224	40,237
786,432	228.938	13.467	10,124,136	44,222
1,048,576	303.870	17.875	13,269,864	43,670
1,310,720	382.158	22.480	16,415,592	42,955
1,572,864	461.488	27.146	19,561,320	42,387

from 49,152 through 24. The common primary metric is total wall divided by eight, 352.145 s/depth; the auxiliary final-depth time was 1,644.13 s.

The output-24 OOM recurred while the runtime descended its row-budget profiles from 2,048 to 1. Its 1.563-s field is the final failed attempt, not a completed depth time.

AlphaCube materializes all $18B$ Cube3 children in host NumPy arrays before sorting; consequently its first failure is a host-memory kill even though GPU reservation is only 4.685 GiB. Its measured rate settles near 40–44 thousand nodes/s at the widest completed points. DeepCubeA has no persistent beam: it is weighted A* and its native scale is search/NN batch 10,000/10,000, so inventing a frontier maximum for it would be incorrect. Its completed T4 run performed 55 search iterations in 146.474 s, or 2.663 s per native A* iteration; this is not a beam-depth measurement.

The same run expanded 6,134,352 nodes at 41,880 nodes/s and returned a length-21 solution.

For the matched scalar Megaminx pair, CayleyPy commit 5b2b53f uses iterated search with `history_depth=0`. The broad protocol completed every power of two from 2¹⁰ through 2²⁴ and CUDA-OOMed at 2²⁵; the memory-trend estimate equalled 2²⁴, so no redundant confirmation was scheduled. Earlier targeted evidence with the same contract retains the finer 20,971,520-complete versus 21,037,056-OOM bracket, but is not presented as part of the broad-grid policy. Its internal expansion path bypassed the public counter hook, so only wall time and memory are valid. The MultiGPU likewise retained end-to-end wall but no valid node counter. The final scalar pass measured 29,360,128 complete at 14,215 MiB versus 31,457,280 CUDA OOM at 14,909 MiB. The matched output-24 pass measured 29,360,128 complete versus 33,554,432 CUDA OOM. Superseded static profile failures and the cancelled v10/v11 runs remain excluded. At the latest broad-grid matched width, CayleyPy took 4,741.581 s and the historical MultiGPU run took 1,728.460 s at $B = 16,777,216$ (2.74x lower). At the older targeted $B = 20,971,520$ point, MultiGPU wall is 1.95x lower. These are matched end-to-end wall ratios, not inferred node-throughput ratios.

The scalar ratios above use one identical scalar model contract and must not be compared with the faster Pilgrim QSearcher row. The 24-output model evaluates all 24 actions per parent and has a different architecture; scalar-versus-Q timing is therefore a model-contract difference, not an algorithmic speedup. The separate Cube4 comparison uses the identical 24-output Q checkpoint for Pilgrim and MultiGPUBeamSearch. No output-24 CayleyPy value

Table 8: Completed native CayleyPy 1xT4 powers-of-two sweep. Memory is the PyTorch CUDA allocator peak; the final row is the first broad-grid failure.

B	Wall (s)	s/depth	Allocated (MiB)	Reserved (MiB)	Status
1,024	0.930	0.116	85.9	92.0	complete
2,048	1.237	0.155	96.3	112.0	complete
4,096	2.189	0.274	117.3	128.0	complete
8,192	3.828	0.478	158.6	230.0	complete
16,384	6.776	0.847	241.5	292.0	complete
32,768	13.722	1.715	407.5	510.0	complete
65,536	31.399	3.925	739.3	858.0	complete
131,072	63.664	7.958	772.2	1,180.0	complete
262,144	114.551	14.319	834.8	1,168.0	complete
524,288	219.991	27.499	962.1	2,156.0	complete
1,048,576	430.789	53.849	1,216.7	3,234.0	complete
2,097,152	745.341	93.168	1,726.0	2,740.0	complete
4,194,304	1,320.000	165.000	2,745.2	4,492.0	complete
8,388,608	2,489.356	311.169	5,155.3	6,462.0	complete
16,777,216	4,741.581	592.698	10,233.7	14,652.0	complete
33,554,432	—	—	12,922.2	14,484.0	CUDA OOM

Table 9: Final MultiGPU 1xT4 scalar boundary points.

B	Wall (s)	s/depth	Peak device (MiB)	Status
25,165,824	3,027.673	378.459	12,267	complete
27,262,976	3,080.206	385.026	13,241	complete
29,360,128	2,817.158	352.145	14,215	complete
31,457,280	—	—	14,909	CUDA OOM

Table 10: Final MultiGPU 1xT4 native output-24 sweep.

B	Wall (s)	s/depth	Peak device (MiB)	Status
1,048,576	8.296	1.037	773	complete
4,194,304	22.735	2.842	2,253	complete
8,388,608	41.091	5.136	4,159	complete
16,777,216	77.460	9.683	7,983	complete
20,971,520	96.124	12.015	9,893	complete
25,165,824	111.130	13.891	11,799	complete
29,360,128	116.904	14.613	13,717	complete
33,554,432	1.563	—	14,909	CUDA OOM

is inferred.

Thus, at the matched Pilgrim point our tenth-layer throughput is $1.093\times$ on T4 and $1.042\times$ on P100. PyTorch allocator reservation and device-wide `nvidia-smi` usage are different memory metrics, so the table does not claim a memory ratio; it establishes that both implementations complete 4M.

For the Megaminx superflip instance, the located public chronology improves from 55 through 54 and 53 to 51 moves; the immediately preceding public value was therefore 53, not 55 [16, 15]. The 51-step path uses the same 24-generator signed 72-degree face-turn metric and has public wall time 22,600.2 s. Independent CPU replay of the recorded submission reaches the exact central state with zero differences; its path and file SHA-256 values are recorded in the accompanying evidence audit. The cited public chronology contains no shorter same-metric path, but no formal registry or optimality proof exists.

Table 11: Matched 1xT4 Megaminx scalar sweep. Neither implementation exposes a valid node counter for these points.

B	MultiGPU		CayleyPy	
	Wall (s)	s/depth	Wall (s)	s/depth
65,536	12.537	1.567	31.399	3.925
262,144	39.935	4.992	114.551	14.319
1,048,576	152.604	19.076	430.789	53.849
4,194,304	476.007	59.501	1,320.000	165.000
8,388,608	892.948	111.619	2,489.356	311.169
16,777,216	1,728.460	216.058	4,741.581	592.698
20,971,520	2,179.990	272.499	4,258.602	532.325
25,165,824	2,416.150	302.019	—	—

We therefore call it the shortest public same-metric solution located, not an unqualified world record. Its missing run manifest is kept separate from the summary-backed 770,883,178-requested-state depth-8 capacity report.

6 Code and Artifacts

Paper sources, the evidence index, and machine-readable tables are public at <https://github.com/TryDotAtwo/MultiGPUBeamSearchPaper>. The baseline v1.0.0 release contains matched checkpoints and earlier PDFs: <https://github.com/TryDotAtwo/MultiGPUBeamSearchPaper/releases/tag/v1.0.0>. Solver source and its GPU-ready Megaminx release are public at <https://github.com/TryDotAtwo/MultiGPUBeamSearch> and <https://github.com/TryDotAtwo/MultiGPUBeamSearch/releases/tag/megaminx-native-cluster-latest>. The public self-contained one-T4 reproduction dataset is <https://www.kaggle.com/datasets/trydotatwo/gpu-beam-search-paper-1xt4-reviewer-notebooks>. Four importable notebook sources for the Pilgrim and MultiGPUBeamSearch broad sweeps and boundary

confirmations, together with CayleyPy, AlphaCube, and DeepCubeA runners, are versioned in the paper repository. They consume the public dataset through Kaggle’s normal Import Notebook and Save Version workflow and do not depend on the visibility of an owner’s saved kernel page. The eight-A100 record supplies capacity and memory evidence but no wall time or exact effective width; the two-T4 record separately supplies measured throughput. The eight-H200 Cube4 run additionally supplies a completed billion-state depth with full rank logs, device telemetry, and an evidence manifest at https://github.com/TryDotAtwo/MultiGPUBeamSearchPaper/tree/main/artifacts/results/h200_8x_cube4. The manifest specifies the implementation, model, and interrupted follow-on depth. No speedup ratio across these different workloads is inferred.

7 Conclusion

This work establishes one GPU-resident distributed beam search rather than a collection of local surrogates. For a fixed layout, its streamed expand–de-duplicate–select path, with no shard-local semantic caps, is mathematically equivalent to monolithic global reduced-key top- B under the same score quantization, tie order, and stated 128-bit identity model. The eight-H200 Cube4 result establishes a physically sharded retained frontier of 2,900,361,216 states and measures one completed saturated depth at 74.746 million nominal parent-action pairs/s end to end. The separate two-T4 Megaminx result measures 30.274 million logical children/s; these different workloads do not establish strong scaling. The eight-A100 capacity-only report has no preserved wall time. The project also provides a replay-valid 51-move Megaminx-superflip path, the shortest public same-metric solution identified in the supporting evidence. This is the first published high-throughput multi-GPU beam-search system with one physically sharded global frontier and the stated exactness contract at billion-state retained-frontier scale.

References

- [1] F. Agostinelli, S. McAleer, A. Shmakov, and P. Baldi. Solving the rubik’s cube with deep reinforcement learning and search. *Nature Machine Intelligence*, 1:356–363, 2019.
- [2] T. S. Anantharaman and R. Bisiani. A hardware accelerator for speech recognition algorithms. In *Proceedings of the 13th Annual International Symposium on Computer Architecture*, pages 216–223, 1986.
- [3] H. Braun, J. Luitjens, R. Leary, T. Kaldewey, and D. Povey. GPU-accelerated viterbi exact lattice decoder for batched online and offline speech recognition. In *2020 IEEE International Conference on Acoustics, Speech and Signal Processing*, 2020.
- [4] W. Chan et al. Efficient beam search for large language models using trie-based decoding. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 14795–14807, 2025.
- [5] A. Chervov et al. A machine learning approach that beats large rubik’s cubes. *arXiv preprint arXiv:2502.13266*, 2025.
- [6] J. Chong, Y. Yi, A. Faria, N. R. Satish, and K. Keutzer. Data-parallel large vocabulary continuous speech recognition on graphics processors. Technical Report UCB/EECS-2008-69, University of California, Berkeley, 2008.
- [7] N. Frohner, J. Gmys, N. Melab, G. R. Raidl, and E.-G. Talbi. Parallel beam search for combinatorial optimization. In *Proceedings of the 51st International Conference on Parallel Processing Workshops*, 2022.
- [8] HATETRIS public beam-search contributors. HATETRIS public beam-search implementation, 2026. Public repository, accessed 25 August 2026.
- [9] M. Junczys-Dowmunt, T. Dwojak, and H. Hoang. Is neural machine translation ready for deployment? a case study on 30 translation directions. In *Proceedings of the 13th International Conference on Spoken Language Translation*, 2016.
- [10] S. Kim, S. Park, S. U. Noh, J. Hong, T. Kwon, H. Lim, and J. Lee. PathWeaver: A high-throughput multi-gpu system for graph-based approximate nearest neighbor search. In *2025 USENIX Annual Technical Conference*, pages 1501–1517, 2025.
- [11] A. Kishimoto, A. Fukunaga, and A. Botea. Evaluation of a simple, scalable, parallel best-first search strategy. *Artificial Intelligence*, 195:222–248, 2013.
- [12] R. Kuroiwa and J. C. Beck. Parallel beam search algorithms for domain-independent dynamic programming. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pages 20743–20750, 2024.
- [13] A. Soibelman et al. CayleyPy RL: Pathfinding and reinforcement learning on cayley graphs. *arXiv preprint arXiv:2502.18663*, 2025.
- [14] M. Soltani and F. Agostinelli. GPU-accelerated A* search with deep neural network heuristics. In *Proceedings of the Nineteenth International Symposium on Combinatorial Search*, 2026.
- [15] SpeedSolving forum contributor. Development of a megaminx solver: 51-move superflip report. SpeedSolving forum post 1705451, 2026.
- [16] SpeedSolving forum contributor. Development of a megaminx solver: 53-move superflip report. SpeedSolving forum post 1700151, 2026.
- [17] D. Sun, B. Kan, H. Long, H. Zhao, H. Li, Y. Liu, P. Zhou, A. Feng, W. Huang, Y. Gu, Z. Li, H. Shang, Y. Zhang, D. Tao, N. Sun, and G. Tan. A fully GPU-accelerated framework for high-performance configuration interaction selection with neural network quantum states. In *Proceedings of the 35th International Symposium on High-Performance Parallel and Distributed Computing*, 2026.
- [18] K. Takano. Self-supervision is all you need for solving rubik’s cube. *Transactions on Machine Learning Research*, 2023.
- [19] A. Wijs and B. Lisser. Distributed extended beam search for quantitative model checking. In *Model Checking Software: 14th International SPIN Workshop*, pages 166–184, 2007.
- [20] A. Wijs and M. Osama. The fast and the capacious: Memory-efficient multi-GPU accelerated explicit state space exploration with GPUexplore 3.0. *Frontiers in High Performance Computing*, 2, 2024.
- [21] Y. Zhou and J. Zeng. Massively parallel A* search on a GPU. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2015.